孪生地球 Twin Earth

UDOS 论文卷 · 旗舰论文完善版

P0b 结构加规模，万物皆可 Scaling Laws（v7.6.2 完善版）

编号：P0b

标题：结构加规模，万物皆可 Scaling Laws

完善版版本：v7.6.2（基于 v7.5.0 原卷升级；DeepSeek《一切皆插件：时空可组合性编程范式》理论注入版）

作者：房文新

归档日期：2026-10-11

证据等级：cpu-proto（UDOS 实测数字分级见文末溯源表；DeepSeek 形式化引用为外部文献级）

P0b 结构加规模，万物皆可 Scaling Laws（v7.6.2 完善版）

孪生地球 Twin Earth · UDOS v7.6.2 · 旗舰论文完善版

- 编号：P0b
- 标题：结构加规模，万物皆可 Scaling Laws
- 完善版版本：v7.6.2（基于 v7.5.0 原卷升级；DeepSeek《一切皆插件：时空可组合性编程范式》理论注入版）
- 作者：房文新
- 归档日期：2026-10-11
- 证据等级：cpu-proto（UDOS 实测数字分级见文末溯源表；DeepSeek 形式化引用为外部文献级）

摘要（本版更新）

原摘要主张：规模的收益是结构的函数；把"结构"与"规模"两个变量分开后，可以画出一条三段式 Scaling 曲线——幂律段（规模红利）、结构相变段（规模开始惩罚，收益拐头）、饱和/负收益段（规模有害），并用七个领域（代谢、城市、智能体吞吐、模型选型、数据飞轮、治理、多视角）的受控对照实证。本完善版在保留全部原论证与实测数字的基础上，注入一份来自 DeepSeek 动态组合演算的规模理论：**组合系统的规模成本不是 O(N) 也不是 $O(N^2)$，而是由依赖图的结构决定的分段函数；生命周期九规则把"组件粒度 × 依赖图 × 运行时状态"这个三元规模问题显式编码，使规模惩罚点可预测、可推迟**。本版据此：

1. 把原"质量饱和/吞吐驼峰"两类相变现象，与"依赖图二次方粒度成本 + 打包/惯例接线/脚手架三类缓解"（DeepSeek §6.5）对应，给出规模曲线的机制解释，而不只是现象描述；

2. 新增预测 F2'：当组件数量增长而依赖图保持分层无环（DAG）时，组合成本保持近线性；依赖图出现循环时，成本进入二次方段——循环依赖（find_cycle 可检测）是结构相变的最早可测信号；

3. 把原"8 个 Agent 反而比 1 个慢"的吞吐驼峰，与"守卫约束下的卸载/重载成本"在机制上对齐：规模惩罚的拐点往往不是算力不足，而是结构维护（守卫检查、逆序回退、依赖广播）的成本接管。

原卷的三段式曲线、七组对照、全部 UDOS 数字逐字保留；新增内容显式标注"v7.6.2 新增"。

关键词：规模；结构；三段式曲线；幂律；结构相变；饱和；吞吐驼峰；质量饱和；动态组合演算；依赖图；粒度成本

1 从三个朴素的现象说起（原卷保留）

1.1 大象为什么不是一千只老鼠

一只大象的体重约为一只老鼠的十万倍，但它的代谢率并不是十万倍，而大约是十万倍的四分之三次方——体重翻倍，维持生命所需的能量只增加约 75%。这是生物学家克莱伯（Kleiber）在 1932 年就发现的规律，后来被称为克莱伯律。

更大的身体不是免费的。血管必须分叉再分叉，才能把养分送到每一个细胞；网络越长，输运的"结构性折扣"就越大。规模增长带来的收益，天然是要打折扣的，折扣率由输运网络的结构决定。

1.2 城市越大越省，也越大越赚

物理学家贝当古（Bettencourt）等人 2007 年分析了大量城市数据，发现一个对称的现象：城市人口翻倍，道路、电缆、加油站这类基础设施只增加约 80%（亚线性，省）；而专利、工资、GDP 这类创新产出却增加约 120%（超线性，赚）。

同一个"人口规模"轴上，不同种类的"结构量"给出了方向相反的指数。这说明规模本身没有固定的好坏，好坏取决于你在数什么、以及这些东西是怎么组织的。

1.3 两个智能体最快，八个反而最慢

在我们自己的引擎上，一个更反直觉的现象直接可复现。让并发智能体处理一批互相独立的任务：

- 1 个智能体时，每秒处理 153 个任务；
- 2 个时达到峰值 244 个（1.59 倍加速）；
- 4 个时回落到 158 个；
- 8 个时只剩 106 个——比 1 个还慢。

但只要把任务类型从"抢 CPU"换成"等待外部输入"（比如等网络、等磁盘），同样从 1 加到 8 个，吞吐几乎完美线性增长到近 8 倍。

规模轴完全相同，唯一改变的是"任务结构与硬件结构是否匹配"。匹配时，加规模近乎免费；错配时，加规模是在给系统添乱。

这三个现象——代谢、城市、智能体——合起来指向本文的中心论点：**规模从来不是好东西，也不是坏东西。规模的收益是结构的函数。**

1.4 v7.6.2 新增：第四个朴素现象——插件越多，未必越快

原卷的 1.3 观察的是"执行任务的数量"；本版补上"组装系统的数量"。DeepSeek 论文记录了一个开放插件生态（Koishi，4000+ 插件，p69）的规模事实：插件数量增长带来功能增益，但组件间依赖关系的数量按"两两组合"的速度增长——N 个插件最坏有 N(N-1)/2 对潜在依赖。如果组合成本随依赖对数量线性增长，那么组件规模一旦过百，纯依赖管理的成本就超过功能收益；这个生态之所以没有在百组件规模崩塌，是因为运行时把依赖维护从"作者自觉"转移给了"结构规则"（九条生命周期规则 + 依赖图 + 守卫，p34-37），把二次方依赖成本压回近线性。

这不是巧合，而是本文三段式曲线的又一个实例：**规模收益曲线在哪一段，取决于你用什么结构去承担规模**。代谢用分叉血管，城市用层级路网，插件生态用生命周期规则——都是"用结构把规模成本打折"。

2 把话说清楚：两个变量、一条曲线（原卷保留）

2.1 结构是什么，规模是什么

沿用姊妹篇 P0 的定义，结构包含三件事：

- 连接拓扑：谁和谁相连、信息往哪个方向流（对应多智能体里的星型/分层/网状）；
- 几何/空间组织：连接在嵌入空间中的度量与方位（扇入扇出、距离、位姿）；
- 模块边界：系统切成几块、每块封装什么、块间交换什么。

规模是另一个独立变量：系统里"多少个"——多少 Agent、多少参数、多少数据、多少组件。规模是数量，结构是形状。

2.2 一条曲线：三段式 Scaling Law

把结构与规模分开后，本文提出三段式曲线（master curve）：

1. **幂律段**：规模红利期。加规模近乎免费（甚至超线性），收益曲线向上凸。对应：小城市的创新、2 个 Agent 的吞吐峰值、参数尚小时模型能力快速爬升。

2. **结构相变段**：规模开始惩罚。收益拐头，曲线变平甚至下弯。原因不是"规模本身有毒"，而是**结构的某个承载量被击穿**——血管总截面积不够、路网节点过载、依赖图出现环、锁竞争加剧。

3. **饱和/负收益段**：规模有害。继续加规模收益为负。对应：8 个 Agent 的吞吐驼峰、同质 Agent 军团的质量饱和、同质化扩招失效。

中心命题：三段式曲线的拐点位置，由结构决定，不由规模本身决定。 改变结构可以把拐点推后（分层 vs 星型），也可以提前（同质化扩招）。

2.3 v7.6.2 新增：三段式曲线在"动态组合演算"中的对应

把"组件数量"当作规模轴、"组合成本"当作收益轴，动态组合演算给出同一条三段式：

- **幂律段（依赖稀疏）**：组件少、依赖图接近森林（无环、低连通），运行时只需做装载检查 O(N)，组合成本近线性；
- **结构相变段（依赖稠密化）**：组件增多、依赖对逼近 N^2 量级，若依赖关系由作者手写注册表管理，组合成本按依赖对线性增长——等价于 $O(N^2)$；运行时若采用"惯例接线 + 打包 + 脚手架"（DeepSeek §6.5 缓解），把显式依赖对压回常量级，相变拐点推迟；
- **饱和/负收益段（循环依赖）**：依赖图出现环（A 依赖 B、B 依赖 A），任何装载顺序都不满足全部依赖；此时组合成本爆炸且不可满足——这对应原卷"质量饱和"的机制版本。

新预测 F2'（v7.6.2 新增，可证伪）：在组件规模 N 增长、功能集保持一致的受控实验中，依赖图为 DAG（分层无环）的系统，其单次装载的平均成本增长阶数 ≤ O(log N)（或近线性常数级）；引入 ≥1 个循环依赖后，同系统的装载成本进入超线性段且出现不可满足装载（装载失败）。反例条件：DAG 系统装载成本出现 O(N) 及以上增长，或循环依赖系统仍保持近线性装载成本。

3 七个领域的受控对照（原卷保留）

原卷的七组对照（此处列其结论与关键数字，详见原卷）：

1. **拓扑对照**：星型扇入 20,971,520/轮次 vs 分层 9/轮次，规模轴相同，结构不同，成本差数百万倍（matrix_scale_demo.json，cpu-proto）。

2. **吞吐驼峰**：CPU 任务 1/2/4/8 Agent = 153.208/244.172/158.386/106.485 任务/秒；I/O 任务 19.94/39.88/79.69/159.26 近线性（agent_scaling.json，cpu-proto，单种子）。

3. **质量饱和**：mse=a+b/k, a=0.009252、b=-4.233×10⁻⁵，best single 0.009196，等权专家 0.2131（agent_scaling.json，k≤2 verified，余 cpu-proto）。

4. **数据飞轮**：良率 0.667→0.854，状态单元 157→215，密度 7.80→11.38，同池贪心消融 0.7917/0.889（egodata_flywheel_demo.json，cpu-proto，48 回合/144 分钟，oracle 226 单元）。

5. **模型选型**：宽度 64/128/256 = 95,348/271,476/967,796 参数（model_size_convergence.json，verified）。

6. **治理规模税**：stigmergy 60 条 vs 合同网 240 条消息，n≥3f+1（udos7/topology/，cpu-proto）。

7. **多视角饱和**：单视角 IoU 0.7147/覆盖 0.7456/precision 0.9472/depth MAE 0.3186；八视角 IoU 0.8897/覆盖 0.9957（spatial_novelview_demo.json，cpu-proto，8 训练视角/6 留出/体素 0.1m）。

3.1 v7.6.2 新增：对照 2/3 的机制解释

原卷把吞吐驼峰与质量饱和作为现象报告；本版给出结构机制的初步解释：

- **吞吐驼峰**：CPU 任务下 2→4→8 的回落，与锁/共享状态竞争、上下文切换成本接管有关——这是"结构（并发访问方式）被规模击穿"的相变实例；I/O 任务无共享状态竞争，因此无相变。机制原型可在 UDOS 内以"共享状态访问图"复现（试点口径，见附录）。
- **质量饱和**：同质 Agent 军团（等权专家）在扩招后 MSE 停留在 0.2131，远劣于最优单模型 0.009196——同构节点无法提供异构信息增益，规模进入负收益段。这与 DeepSeek 的"组件粒度与依赖类型化"对应：**一味加同质组件，等于给系统加依赖而不加信息**；结构上引入异质（分层、分型）才是推迟饱和的路径。

4 统一：三段式曲线的结构参数化（v7.6.2 增补，原卷 4-5 章保留）

4.1 原卷统一（保留）

原卷第 4-5 章论证：代谢（分叉网络）、城市（层级路网）、智能体（分层混合拓扑）共享同一数学形态——**规模收益指数 = 结构量的指数**。数"输运量"得到亚

线性，数"创新量"得到超线性；数"任务吞吐"得到驼峰，数"I/O 吞吐"得到近线性。同一个 N，不同的"数的对象"，不同的指数——对象由结构决定。

4.2 v7.6.2 新增：动态组合演算的规模方程

把 DeepSeek 的动态组合演算纳入统一：组合系统的总成本 C(N, G) 可分解为

C(N, G) = C_load(N, G) + C_guard(G) + C_revert(G) + C_notify(G)

其中 N 为组件数，G 为依赖图结构：

- C_load：装载/实例化成本，近线性（幂律段）；
- C_guard：守卫检查成本（卸载/重载前遍历依赖者，Def54 $relied_n$），依赖图稀疏时近常数，稠密时随依赖对增长（相变段）；
- C_revert：逆序回退成本（可逆效应链，p9-16），正比于效应链深度——链深由"每步写多少状态"决定，是结构量；
- C_notify：余效应广播成本（notify_d，p18），正比于依赖者数量——结构量。

结论（v7.6.2 表述）：三段式曲线的拐点 = C_guard + C_revert + C_notify 三项结构成本追上 C_load + 功能收益的时刻。这给出一个可操作的工程判据：**推拐点 = 减依赖对（打包/分层）+ 减链深（快照/检查点）+ 减广播面（主题订阅/白名单）**——与代谢（减输运距离）、城市（减基础设施重复）的策略完全同构。

5 最严厉的反对意见（原卷保留 + v7.6.2 增补）

5.1 原卷反对意见（保留）

- "三段式只是拟合，不是定律"——本文以七组受控对照 + 明确反例条件回应；
- "克莱伯律 3/4 只是经验近似"（Hulbert 等）——本文引用其为现象起点，不依赖其精确指数；
- "Dunbar 数因果方向存疑"——谨慎引用；
- "涌现只是度量选择"（Schaeffer 等 2023）——本文不用涌现术语。

5.2 v7.6.2 增补：对"动态组合演算只是单生态证据"的回应

与 P0 v7.6.1 一致：DeepSeek 的生态观测是单生态、单语言、存在性的（p70），其规模结论（组合成本、缓解手段）来自一个具体运行时。本版把其机制（C_load/C_guard/C_revert/C_notify 分解）作为**假设**用于解释 UDOS 对照 2/3，并

明确标注：该分解在 UDOS 上的成立性以 4.4/3.1 的机制原型为准（cpu-proto 试点），不声称已在真实开源生态定量验证。F2' 是待证伪的预测，不是已证结论。

6 结语（v7.6.2 更新）

规模从来不是好东西，也不是坏东西。规模的收益是结构的函数——这句话在 v7.5.0 由七个领域的对照支撑，在 v7.6.2 又获得了一层工程印证：一个四千插件的生态能撑住规模，不是因为插件写得好，而是因为生命周期规则把"依赖维护成本"从作者自觉转移给了结构。代谢用分叉，城市用层级，插件生态用九条规则——**每次规模跨越，都是一次结构对规模成本的重新打折**。把"规模"与"结构"分开，把曲线画成三段，把拐点变成可操作的结构参数，是本文与 DeepSeek 研究共同的工程信条。

参考文献（本版新增条目以 ▲ 标注）

复用本卷《参考文献总库》（2026-09-19 核验），原卷 17 条保留（West 1997、Kleiber 1932/1961、Bettencourt 2007、Herculano-Houzel 2012、Amdahl 1967、Gunther 2008、Dennard 1974、Kaplan 2020、Hoffmann 2022、Fedus 2021、梁文锋 2025、Schaeffer 2023、Frankle 2019、Briscoe 2006、Dunbar 1990s、Vaswani 2017、Smith 1980），另：

18. ▲ Shi, Y., Zhang, W., Cui, T. (2025). A Programming Paradigm for Spatiotemporal Composability (预印本 v1；DeepSeek-AI / 北京大学). 本文引用其 §6.5 粒度成本与缓解、p34-37 生命周期规则、p17-20 余效应语境 [CITATION NEEDED：正式出版版本号]。

[CITATION NEEDED]：电芯组规模标度的同行评议实测指数；Dennard 1974 与 Dunbar 原始论文的精确卷页。

附录：本文 UDOS 数字溯源表

数字	数据文件	等级
---	---	---
扇入 20,971,520→9、轮次 8,388,608→14（六档）	matrix_scale_demo.json	cpu-proto（前两档实测）
CPU 吞吐 153.208/244.172/158.386/106.485；I/O 19.94/39.88/79.69/159.26 近线性	agent_scaling.json	cpu-proto（单种子）
质量饱和 a=0.009252, b=-4.233×10⁻⁵，best single 0.009196；等权 0.2131	agent_scaling.json	k≤2 verified，余 cpu-proto

数字	数据文件	等级
良率 0.667→0.854；状态单元 157→215；密度 7.80→11.38；同池贪心消融 0.7917/0.889	egodata_flywheel_demo.json	cpu-proto（48 回合/144 分钟，oracle 226 单元）
宽度 64/128/256 = 95,348/271,476/967,796 参数	model_size_convergence.json	verified
stigmergy 60 条 vs 合同网 240 条：n≥3f+1	udos7/topology/	cpu-proto
多视角单视角 IoU 0.7147/覆盖 0.7456/precision 0.9472/depth MAE 0.3186；八视角 IoU 0.8897/覆盖 0.9957	spatial_novelview_demo.json	cpu-proto（8 训练视角/6 留出/体素 0.1m）
▲ 组合成本分解 C_load/C_guard/C_revert/C_notify 机制原型（DAG 近线性 vs 循环依赖不可满足）	udos7/topology/composability_demo（v7.6.2 新增）	cpu-proto 试点估计（单 seed）

互链：本文为"规模轴"。结构轴见 P0，自我边界轴见 P0c，群体智能轴见 P0d。